

Prof. João Paulo Biazotto

Engenharia de Software

Objetivos da aula

- Compreender o propósito de um Diagrama de Sequência.
- Identificar atores, objetos, lifelines e mensagens.
- Representar chamadas, retornos, condições e repetições.
- Relacionar o Diagrama de Sequência com o Diagrama de Classes.
- Construir um diagrama a partir de uma história de usuário.

O que é um Diagrama de Sequência?

Ideia principal

O Diagrama de Sequência mostra **como os participantes de um cenário interagem ao longo do tempo**.

Ele ajuda a responder perguntas como:

- Quem inicia a interação?
- Quais objetos participam?
- Qual mensagem é enviada primeiro?
- Qual objeto executa cada responsabilidade?
- Existem caminhos alternativos?
- Existe repetição?

Classes × Sequência

Diagrama de Classes

Mostra a **estrutura** do sistema.

- Classes
- Atributos
- Métodos
- Relacionamentos

Diagrama de Sequência

Mostra a **interação** em um cenário.

- Participantes
- Ordem temporal
- Mensagens
- Condições

Classes mostram quem existe. Sequência mostra como eles colaboram.

Cenário que iremos modelar

Sistema de Streaming Musical

Um usuário escolhe uma música e solicita sua reprodução. O sistema verifica se a assinatura está válida. Se estiver válida, a música é reproduzida e o sistema registra a reprodução.

Pergunta inicial:

Quem participa desse cenário?

Participantes do cenário

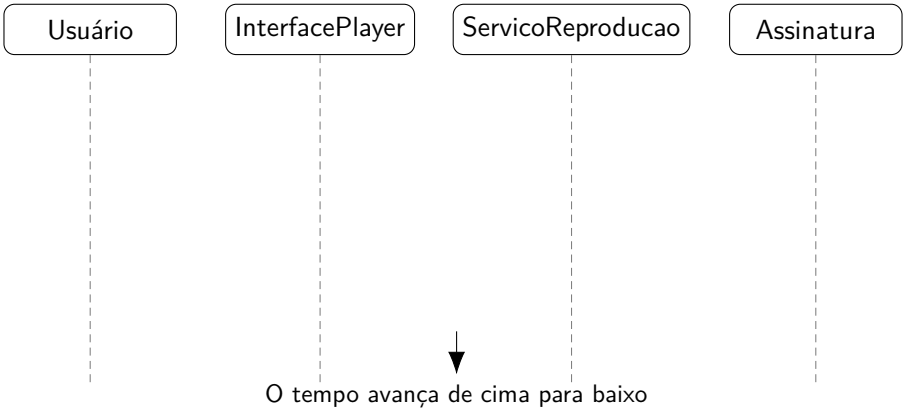
Possíveis participantes:

- **Usuário** – ator que inicia a interação;
- **InterfacePlayer** – interface utilizada pelo usuário;
- **ServicoReproducao** – coordena o caso;
- **Assinatura** – verifica se o acesso é permitido;
- **Musica** – conteúdo a ser reproduzido;
- **Reproducao** – registra o histórico.

Importante

Nem todo participante precisa ser uma entidade de domínio. Podemos ter ator, interface, controle/serviço e entidades.

Lifelines



Lifeline

A linha vertical representa a existência do participante ao longo do cenário.

Mensagens

Primeira interação

O usuário solicita a reprodução de uma música.

```
Usuario -> InterfacePlayer: reproduzirMusica(idMusica)
```

Depois, a interface delega a responsabilidade:

```
InterfacePlayer -> ServicoReproducao:
    reproduzir(idUsuario, idMusica)
```

Pergunta importante

Qual objeto deve receber a responsabilidade por cada operação?

Chamadas e retornos

Chamada:

```
ServicoReproducao -> Assinatura:
    estaValida()
```

Retorno:

```
Assinatura --> ServicoReproducao:
    true
```

Convenção

- Seta contínua: chamada.
- Seta tracejada: retorno.

O retorno pode ser omitido quando o resultado estiver claro no contexto.

Construindo o cenário passo a passo

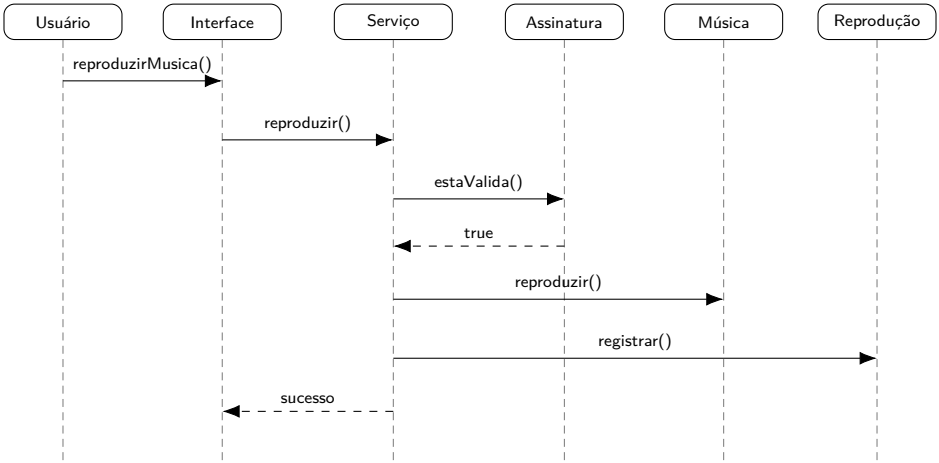
1. Usuário solicita a reprodução.
2. Interface delega ao serviço.
3. Serviço verifica a assinatura.
4. Música é reproduzida.
5. Reprodução é registrada.
6. Resultado retorna ao usuário.

Fluxo

$$\text{Usuário} \rightarrow \text{Interface} \rightarrow \text{Serviço} \rightarrow \text{Assinatura}$$

$$\text{Serviço} \rightarrow \text{Música} \rightarrow \text{Reprodução}$$

Diagrama básico do cenário



Condições com `alt`

Pergunta:

E se a assinatura não estiver válida?

```
alt assinatura valida
    ServicoReproducao -> Musica: reproduzir()
    ServicoReproducao -> Reproducao: registrar()
else assinatura invalida
    ServicoReproducao --> InterfacePlayer: acessoNegado
    InterfacePlayer --> Usuario: assinatura invalida
end
```

Interpretação

O fragmento `alt` representa caminhos alternativos, semelhante a um `if/else`.

Comportamento opcional com opt

Quando uma interação pode ou não ocorrer:

```
opt registrar historico
    ServicoReproducao -> Reproducao: registrar()
end
```

opt

Representa um comportamento opcional, executado somente quando determinada condição for satisfeita.

Repetição com `loop`

Considere agora a reprodução de uma playlist:

```
loop para cada musica da playlist
    ServicoReproducao -> Musica: reproduzir()
end
```

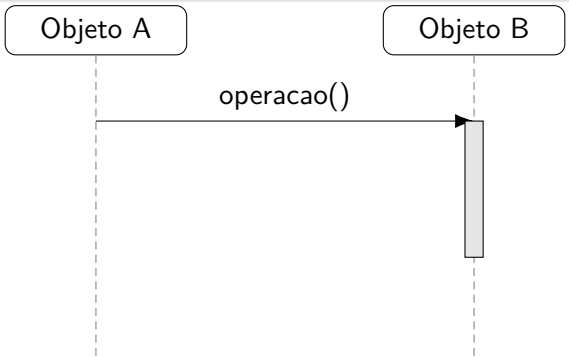
`loop`

Representa um conjunto de mensagens que pode ser executado repetidamente.

Barras de ativação

Ativação

A barra de ativação representa o período em que um participante está executando uma operação.



Criação e destruição de objetos

Criação

Nem todos os objetos precisam existir desde o início do cenário.

```
ServicoReproducao -> Reproducao: criar()
```

A lifeline de `Reproducao` pode começar nesse ponto.

Destruição

Quando um objeto deixa de existir durante o cenário, o final da lifeline pode ser representado por um **X**.

Relação com o Diagrama de Classes

Considere a classe:

```
Musica
+ reproduzir()
```

No Diagrama de Sequência:

```
ServicoReproducao -> Musica: reproduzir()
```

Coerência entre diagramas

Uma mensagem enviada a um objeto normalmente corresponde a uma operação disponível na classe daquele objeto.

Pergunte sempre:

Esse método existe na classe correta?

Um exemplo problemático

```
Usuario -> Sistema: tocar musica
Sistema -> Sistema: verificar tudo
Sistema -> Sistema: processar
Sistema -> Usuario: pronto
```

Problemas:

- "Sistema" está genérico demais;
- não conseguimos identificar responsabilidades;
- as interações entre objetos desaparecem;
- é difícil relacionar o diagrama com as classes.

Uma versão melhor

```
Usuario -> InterfacePlayer: reproduzirMusica()

InterfacePlayer -> ServicoReproducao: reproduzir()

ServicoReproducao -> Assinatura: estaValida()

ServicoReproducao -> Musica: reproduzir()

ServicoReproducao -> Reproducao: registrar()
```

Agora conseguimos identificar:

- quem inicia a operação;
- quem coordena o cenário;
- quem possui cada responsabilidade.

Método para construir um Diagrama de Sequência

1. Escolha um **cenário específico**.
2. Identifique os **participantes**.
3. Identifique quem **inicia** a interação.
4. Liste as mensagens na **ordem temporal**.
5. Distribua as responsabilidades entre os objetos.
6. Identifique **condições**.
7. Identifique **repetições**.
8. Compare com o **Diagrama de Classes**.

Perguntas que ajudam durante a modelagem

- Quem recebe essa responsabilidade?
- Qual objeto deveria executar essa operação?
- Esse método existe nessa classe?
- Essa mensagem acontece sempre?
- Existe alguma condição?
- Existe repetição?
- Esse participante é realmente necessário?
- O cenário está específico o suficiente?

Exemplo em PlantUML

```
@startuml
actor Usuario

participant InterfacePlayer
participant ServicoReproducao
participant Assinatura
participant Musica
participant Reproducao

Usuario -> InterfacePlayer: reproduzirMusica(idMusica)
InterfacePlayer -> ServicoReproducao: reproduzir(idUsuario,idMusica)
ServicoReproducao -> Assinatura: estaValida()

alt assinatura valida
    ServicoReproducao -> Musica: reproduzir()
    ServicoReproducao -> Reproducao: registrar()
else assinatura invalida
    ServicoReproducao --> InterfacePlayer: acessoNegado
end
@enduml
```

Exercício guiado

Construa um Diagrama de Sequência para:

Adicionar uma música a uma playlist

Identifique:

- ator;
- interface;
- objetos envolvidos;
- mensagens;
- retornos;
- condições.

Considere a condição:

A música já pertence à playlist?

Aplicação no Projeto PBL

Escolha uma história de usuário já registrada no Trello.
Exemplo:

História de usuário

Como usuário, quero adicionar uma música a uma playlist para organizar minhas músicas favoritas.

A partir dela:

1. identifique o cenário;
2. identifique os participantes;
3. determine a sequência de mensagens;
4. identifique condições;
5. verifique a relação com o Diagrama de Classes;
6. construa o Diagrama de Sequência.

Da História de Usuário ao Diagrama

História de Usuário

↓

Cenário

↓

Objetos participantes

↓

Mensagens e condições

↓

Diagrama de Sequência

Resumo

Ideia central

Diagrama de sequência =
quem conversa com quem, em qual ordem, para realizar um cenário.

Classes mostram quem existe.

Sequência mostra como eles colaboram.

Referências

- OMG. *Unified Modeling Language (UML) Specification*.
- SOMMERVILLE, Ian. *Software Engineering*.
- PRESSMAN, Roger S.; MAXIM, Bruce R. *Software Engineering: A Practitioner's Approach*.